

Αλγοριθμικές Τεχνικές για Δεδομένα Ευρείας Κλίμακας

ΑΜ: 4940 ΦΟΥΡΚΙΩΤΗΣ ΑΘΑΝΑΣΙΟΣ

Εισαγωγή

Η παρούσα εργασία αφορά αλγοριθμικές τεχνικές για την επεξεργασία δεδομένων μεγάλης κλίμακας κυρίως σε αλγορίθμους που χρησιμοποιούν μόνο λίγη πληροφορία άρα και μικρότερο σφάλμα για να κάνουν κερδίσουν σε μνήμη και επικοινωνία.

Η εργασία οργανώνεται σε τρεις ενότητες. Η Ενότητα Α μελετά τους μετρητές Morris, προσεγγιστική μέτρηση πλήθους στοιχείων με πολύ λίγη μνήμη. Η Ενότητα Β ασχολείται με την εκτίμηση του πλήθους διακεκριμένων στοιχείων, μέσω των αλγορίθμων Flajolet-Martin και BJKST. Τέλος, η Ενότητα Γ εξετάζει τα φίλτρα Bloom για τομή συνόλων και για packet tracing σε γράφους και δίκτυα .

Για κάθε ενότητα παρέχεται: (α) πλήρης υλοποίηση σε Python, (β) ανάλυση του κώδικα και της λογικής πίσω από αυτόν , (γ) πειραματικά αποτελέσματα με plots.

Ενότητα Α: Μετρητές Morris

Από την εκφώνηση ζητείται να υλοποιήσω τον μετρητή Morris. Για την περίπτωση $\alpha = 2$, σε κάθε εισαγωγή ο μετρητής αυξάνει την εσωτερική τιμή C με πιθανότητα $1/(2^C)$, ενώ η εκτίμηση που επιστρέφει είναι $2^C - 1$. Η λογική είναι ότι δεν μετράμε ακριβώς κάθε στοιχείο όπως ένας κοινός counter , αλλά κρατάμε έναν πολύ μικρό μετρητή που αυξάνεται όλο και πιο σπάνια όσο το C μεγαλώνει . Με αυτόν τον τρόπο μπορούμε να εκτιμήσουμε πολύ μεγάλα πλήθη χρησιμοποιώντας πολύ λιγότερη μνήμη από έναν κλασικό counter αφού στην συγκεκριμένη περίπτωση για $n=1.000.000$ αρκούν περίπου 5 bits ενώ χρειάζονται 20 bits για έναν κλασικό counter.

Άσκηση 1α — Βασικός Μετρητής Morris ($\alpha=2$)

Ανάλυση κώδικα

Ο αλγόριθμος υλοποιείται με δύο συναρτήσεις:

```

1 import random
2 import math
3 import os #gia ton fakelo pou tha apothikevw ta plots
4 import sys
5 import matplotlib #gia na sxediazw to kathe plot
6
7 matplotlib.use('Agg') #gia na swsw to plot se arxeio
8 import matplotlib.pyplot as plt
9
10 random.seed(42)
11 os.makedirs('plots', exist_ok=True) #gia kathe run na pairnw thn idia seira random arithmwv oste to plot na vgenei idio an to ksanakanw run
12 #dhmiourgw fakelo plots an den yparxei idi
13
14 def morris_insert(C, alpha=2.0):
15     pithanotita = 1.0 / (alpha ** C)
16     if random.random() < pithanotita:
17         return C + 1
18     return C
19
20 def morris_query(C, alpha=2.0):
21     if alpha == 2.0:
22         return (2.0 ** C) - 1.0
23     return (1.0 / (alpha - 1.0)) * ((alpha ** C) - 1.0)
24
25 # --- Ala ---
26 print("=== Ala: Morris Counter α=2, n=1,000,000 ===")
27 n = 1_000_000
28 alpha = 2.0
29 C = 0
30
31 x_values = [] #gia to grafima
32 y_values = [] #gia ti grafima
33 for i in range(1, n + 1):
34     C = morris_insert(C, alpha)
35     ektimisi = morris_query(C, alpha)
36     print(f"{i}: {ektimisi:.0f}")
37     if i % 10_000 == 0:
38         x_values.append(i)
39         y_values.append(ektimisi)
40     if i % 100_000 == 0:
41         sys.stdout.flush()
42
43 plt.figure(figsize=(10, 5))
44 plt.step(x_values, y_values, where='post', color='steelblue')
45 plt.xlabel('n')
46 plt.ylabel('Estimate')
47 plt.title('Ala: Morris Counter Estimate vs n (α=2)')
48 plt.tight_layout()
49 plt.savefig('plots/Ala_morris.png')
50 plt.close()
51 print("Saved plots/Ala_morris.png")
52

```

Η `morris_insert` αυξάνει τον C κατά 1 με πιθανότητα $1/\alpha^C$. Όσο μεγαλύτερος ο C , τόσο πιο σπάνια θα έχω αύξηση επομένως ο C αυξάνεται πολύ πιο αργά από το πραγματικό πλήθος των n . Η `morris_query` επιστρέφει $(\alpha^C - 1)/(\alpha - 1)$, που είναι η αμερόληπτη εκτίμηση: $E[\text{query}(C)] = n$. Αυτό που περιμένω από την γραφική παράσταση είναι ότι για $n=1.000.000$ και $\alpha=2$, η εκτίμηση δεν θα είναι ακριβής, αλλά θα δίνει μια προσεγγιστική εικόνα με κάποια απότομη αύξηση. Έτσι κάθε φορά που ο C αυξάνεται η εκτίμηση κάνει ένα απότομο άλμα. Στο ερώτημα αυτό ο Morris counter για $\alpha=2$ εκτελέστηκε για 1000000 εισαγωγές. Το γράφημα μας δείχνει την εκτίμηση του μετρητή καθώς αυξάνεται το πλήθος των εισαγωγών. Παρατηρώ ότι η καμπύλη δεν είναι ομαλή αλλά έχει μορφή σκαλοπατιών. Αυτό συμβαίνει επειδή η τιμή C δεν αυξάνεται με κάθε εισαγωγή, αλλά μόνο με πιθανότητα $1/2^C$. Όταν λοιπόν το C αυξηθεί, η εκτίμηση $2^C - 1$ κάνει απότομο άλμα..

Άσκηση 1β — 5 Ανεξάρτητοι Μετρητές: Μέσος Όρος vs Διάμεσος

Ανάλυση κώδικα:

```

53 # — A1β —
54 print("\n===== A1β: 5 independent Morris counters, mean & median =====")
55 import statistics # για tin diameso
56
57 n = 1_000_000
58 alpha = 2.0
59 Cs = [0, 0, 0, 0, 0]
60 x_values_b = []
61 mesos_oros_b = []
62 diamesos_b = []
63
64 for i in range(1, n + 1):
65     new_counters=[]
66
67     for c in counters:
68         new_counters.append(morris_insert(c))
69
70
71     counters=new_counters
72     estimates=[]
73     for c in counters:
74         estimates.append(morris_query(c))
75
76     mesos_oros_est = sum(ektimisi) / 5
77     diamesos_est = statistics.median(estimates)
78     print(i, mean_est, median_est)
79
80     if i % 10_000 == 0: #kratima shmeiwn gia plot gia kathe 10.000 vimata
81         x_values_b.append(i)
82         mesos_oros_b.append(mesos_oros_est)
83         diamesos_b.append(diamesos_est)
84
85 plt.figure(figsize=(10, 5))
86 plt.plot(x_values_b, mesos_oros_b, label='Mean')
87 plt.plot(x_values_b, diamesos_b, label='Median')
88 plt.xlabel('n')
89 plt.ylabel('Estimate')
90 plt.title('"5 Morris counters"')
91 plt.legend()
92 plt.tight_layout()
93 plt.savefig('plots/A1b_mean_median.png')
94 plt.close()
95 print("Saved plots/A1b_mean_median.png")
96

```

Κάθε μετρητής ενημερώνεται ξεχωριστά με τη δική του κλήση της `morris_insert`. Έτσι προκύπτουν 5 διαφορετικές εκτιμήσεις για το ίδιο πλήθος εισαγωγών. Στη συνέχεια υπολογίζω δύο συνολικές εκτιμήσεις: τον μέσο όρο και τη διάμεσο των 5 τιμών, ώστε να δω ποια από τις δύο μεθόδους δίνει πιο σταθερή συμπεριφορά. Συγκρίνω τώρα τον μέσο όρο και την διάμεσο χρησιμοποιώντας το γράφημα. Έτσι φαίνεται ότι η διάμεσος είναι γενικά πιο σταθερή από τον μέσο όρο. Ο λόγος είναι ότι αν ένας από τους 5 μετρητές δώσει πολύ μεγαλύτερη τιμή από τους υπόλοιπους, τότε ο μέσος όρος επηρεάζεται πιο έντονα. Αντίθετα, η διάμεσος επηρεάζεται λιγότερο από τέτοιες ακραίες τιμές και έτσι η καμπύλη της είναι πιο ομαλή. Συμπερασματικά η χρήση 5 ανεξάρτητων μετρητών μειώνει τη διακύμανση σε σχέση με έναν μόνο Morris counter. Στο συγκεκριμένο run η διάμεσος είναι πιο ομαλή, αλλά ο μέσος όρος δίνει καλύτερη τελική προσέγγιση του 1.000.000. Άρα η διάμεσος φαίνεται πιο σταθερή, ενώ ο μέσος όρος πιο ακριβής στο τέλος.

Άσκηση 1γ — Ανάλυση Μνήμης: Πότε Αξίζουν 5 Μετρητές;

Ανάλυση κώδικα:

```

97 # — Aly —————
98 print("\n=== Aly: Memory analysis ===")
99
100 def bits_for_direct(n):
101     return math.ceil(math.log2(n + 1))
102
103 def bits_for_5_morris(n):
104     c_approx = math.ceil(math.log2(n)) # giati gia a=2 isxuei C ≈ log2(n)
105     bits_one = math.ceil(math.log2(c_approx + 1))
106     return 5 * bits_one, c_approx, bits_one
107
108 values = [1_000_000, 10_000_000, 33_554_432, 100_000_000, 1_000_000_000]
109
110 for n in values:
111     morris_bits, c_approx, bits_one = bits_for_5_morris(n)
112     direct_bits = bits_for_direct(n)
113
114     if morris_bits < direct_bits:
115         result = "worthwhile"
116     else:
117         result = "not worthwhile"
118
119     print(f"n={n}")
120     print(f"approx C = {c_approx}")
121     print(f"bits for one Morris counter = {bits_one}")
122     print(f"bits for 5 Morris counters = {morris_bits}")
123     print(f"bits for direct counter = {direct_bits}")
124     print(f"result: {result}")
125

```

Για να αξίζουν οι 5 Morris Counter, πρέπει : $5 \times \lceil \log_2(\log_2(n)+1) \rceil < \lceil \log_2(n+1) \rceil$. Αυτό ισχύει για $n > 2^{25}$ δηλαδή για $n > 33.500.000$.

Για $\alpha=2$ έχω εκτίμηση $2^{\alpha C} \approx n$. Άρα για να φτάσω μέχρι n θέλω $2^{\alpha C} \approx n$ δηλαδή $C \approx \log_2(n)$ Δηλαδή ο εσωτερικός counter C δεν μεγαλώνει σαν n αλλά σαν $\log_2(n)$

Οπότε ένας Morris counter χρειάζεται $\log_2(\log_2(n)+1)$ και οι 5 Morris counter: $5 \times \log_2(\log_2(n)+1)$

n	5 Morris (bits)	Κανονικός counter (bits)	Αποτέλεσμα
1.000.000	25	20	Δεν αξίζει
10.000.000	25	24	Δεν αξίζει
33.554.432	25	26	Αξίζει
100.000.000	25	27	Αξίζει
1.000.000.000	25	30	Αξίζει

Συμπερασματικά καταλήγω ότι για μικρότερα μεγέθη ροής, όπως $n = 1.000.000$ ή $n = 10.000.000$, οι 5 Morris counters δεν με βοηθούν ώστε να έχω κέρδος μνήμης σε σχέση με έναν ακριβή μετρητή. Αντίθετα, από περίπου $n = 2^{25}$ και πάνω αρχίζουν να συμφέρουν, γιατί ο direct counter χρειάζεται περισσότερα bits, ενώ οι 5 Morris counters παραμένουν στα 25 bits. Άρα το πλεονέκτημά τους φαίνεται κυρίως σε πολύ μεγάλα n .

Άσκηση 1δ — 8-bit Morris Counter: Εύρεση Βέλτιστου α

Για το ερώτημα αυτό ουσιαστικά σκέφτομαι ότι αν ο Morris counter έχει 8 bits, δηλαδή ο C μπορεί να πάει από 0 μέχρι 255, ποιο α είναι καλό να διαλέξω;

Μέχρι τώρα δηλαδή για $\alpha = 2$ και για $n = 1.000.000$, ο counter δεν φτάνει πολύ ψηλά. Φτάνει περίπου στο $C \approx \log_2(1.000.000) \approx 20$. Οπότε παρόλο που έχω αρκετό χώρο(μπορώ να κρατήσω τιμές έως 255) πρακτικά χρησιμοποιώ περίπου 20. Άρα τα 8 bits είναι πολλά για $\alpha = 2$. Αυτό αποδεικνύεται από το : $E[C] \approx \log_2(10^6) \approx 19.93$

→ χρειάζονται $\text{ceil}(\log_2(20+1)) = 5$ bits . Άρα μένουν 3 bits αδρανή, τα οποία μπορώ να αξιοποιήσω επιλέγοντας μικρότερο α για καλύτερη ακρίβεια.

Θεωρητική εύρεση βέλτιστου α

```
126 # — A15 —
127 print("\n=== A15: 8-bit Morris counters with different  $\alpha$  values ===")
128 n = 1_000_000
129 max_C = 255 # 8 bits ara mporw na apothikevw times apo 0 ews 255
130 alphas = [1.02, 1.0427, 1.5, 2.0, 3.0]
131
132 plt.figure(figsize=(12, 6))
133 for alpha in alphas:
134     C = 0
135     x_values = []
136     y_values = []
137
138     for i in range(1, n + 1):
139         p = 1 / (alpha ** C)
140
141         if C < max_c and random.random() < p:
142             C += 1
143
144         estimate = morris_query(C, alpha)
145
146         if i % 10000 == 0:
147             x_values.append(i)
148             y_values.append(estimate)
149
150     plt.plot(x_values, y_values, label=f" $\alpha={alpha}$ ")
151     plt.axhline(1_000_000, linestyle="--", label="true n")
152     plt.xlabel("n")
153     plt.ylabel("Estimate")
154     plt.title("A15: 8-bit Morris Counter with Different  $\alpha$  Values")
155     plt.legend()
156     plt.tight_layout()
157     plt.savefig("plots/A15_8bit.png")
158     plt.close()
```

Άρα θέλουμε ο μετρητής να χρησιμοποιεί όλο το εύρος [0..255]. Αν η εκτίμηση στο $C=255$ είναι $n=1.000.000$, τότε ο μετρητής κινείται ομαλά σε όλο το εύρος των 8 bits ενώ το n αυξάνεται από 0 έως 1.000.000. Έτσι συμπερασματικά το βέλτιστο είναι $\alpha \approx 1.0427$. Με $\alpha=2$ ο C φτάνει μόνο στο ~ 20 (σπατάλη bits). Με α πολύ κοντά στο 1 (π.χ. 1.02) ο C κολλάει στο 255 πριν $n=1.000.000$, δίνοντας κατώτερη εκτίμηση. Οπότε το $\alpha=1.0427$ ισορροπεί ανάμεσα και στις δύο περιπτώσεις.

Άσκηση 1ε — 15 Bits: Monte Carlo Σύγκριση Δύο Αλγορίθμων

Στο ερώτημα αυτό συγκρίνονται δύο διαφορετικοί τρόποι χρήσης 15 bits μνήμης.

Ανάλυση κώδικα:

```
170 def run_algo1(n): #Enas Morris counter  $\alpha=2$ , 15 bits ( $C \leq 32767$ ).
171     C = 0
172     max_C = 32767 # o eswterinos counter ftanei mexri ton arithmo  $2^{15}-1=32767$ 
173     for x in range(n):
174         if random.random() < 1.0 / (2.0 ** C):
175             C = min(C + 1, max_C) #gia na ayksanw to c kata 1 an petyxei h pithanothta alla na mhn pernaei to c_max
176     return morris_query(C, 2.0) #gia na metatrepsw to c se ektimisi
177
```

Ο πρώτος αλγόριθμος χρησιμοποιεί έναν μόνο Morris counter με $\alpha = 2$ και επιτρέπει στην εσωτερική τιμή C να φτάσει μέχρι το 32767.

```

178 def run_algo2(n, alpha): #3 Morris counters, 5-bit o kathenas (C ≤ 31) o max_c=31
179     Counters = [0, 0, 0]
180     max_C = 31
181     for y in range(n):
182         for j in range(3): # gia kathe eisagwgh enhmerwnw kai tous 3 morris counters
183             if random.random() < 1.0 / (alpha ** Counters[j]):
184                 Cs[j] = min(Counters[j] + 1, max_C) #ayksanw tov Cs[j] κατά 1 alla oxi panw apo to 31
185             e1 = morris_query(counters[0], alpha)
186             e2 = morris_query(counters[1], alpha)
187             e3 = morris_query(counters[2], alpha)
188
189     return (e1 + e2 + e3) / 3
190

```

Ο δεύτερος αλγόριθμος χρησιμοποιεί τρεις ανεξάρτητους Morris counters, όπου κάθε ένας έχει 5 bits και άρα μπορεί να φτάσει μέχρι το 31. Έτσι η τελική εκτίμηση του δεύτερου αλγορίθμου μας δίνεται ως ο μέσος όρος των τριών εκτιμήσεων που υλοποιήθηκαν.

Για τον δεύτερο αλγόριθμο χρειάζεται να επιλεγεί κατάλληλη τιμή του α.

Θέλω όταν ο πρώτος counter φτάσει κοντά στο 31, η εκτίμησή του να μπορεί να καλύψει το μέγεθος 1,000,000. Γι' αυτό θα λύσω με βάση: $\alpha^{31} \approx 1,000,000$

Αρχικά χρησιμοποιείται η προσέγγιση $\alpha^{31} \approx 1.000.000$, ώστε ένας 5-bit counter να μπορεί να καλύψει περίπου το επιθυμητό μέγεθος. Στη συνέχεια ο κώδικας μου δοκιμάζει μερικές πιθανές τιμές του α και με 50 δοκιμές Monte Carlo για καθεμία επιλέγει εκείνη που δίνει το καλύτερο ποσοστό επιτυχίας

```

191 #gia thn algo2 poio a doulevei kalitera se sxesi me ta ypolloipa
192 alpha_candidates = [2.5, 2.7, alpha_target, 3.0, 3.5]
193 print(" Selecting best α for Algo2 (50 trials each):")
194 best_alpha2 = None
195 best_success_rate2 = -1
196 for a in alpha_candidates:
197     successes = 0
198     for t in range(50):
199         teliki_ektimisi = run_algo2(n_inserts, a)
200         if target_low <= teliki_ektimisi <= target_high:
201             successes += 1
202     pososto = successes / 50
203     if pososto > best_success_rate2:
204         best_success_rate2 = pososto
205         best_alpha2 = a
206 print(f" Selected α={best_alpha2:.4f} for Algo2")
207

```

.Αφού επιλεγεί το κατάλληλο α για τον δεύτερο αλγόριθμο, εκτελούνται 500 δοκιμές Monte Carlo και για τους δύο αλγορίθμους. Σε κάθε δοκιμή ελέγχεται αν η τελική εκτίμηση μετά από 1.000.000 εισαγωγές βρίσκεται στο διάστημα [800.000, 1.200.000]. Με αυτόν τον τρόπο εκτιμάται η πιθανότητα επιτυχίας κάθε αλγορίθμου.

```

207 # Monte Carlo gia sygkrisi poio apo algo1 h algo2 exei kaliteri ektimisi
208 successes_algo1 = []
209 successes_algo2 = []
210 for t in range(n_epanalipseis):
211     est1 = run_algo1(n_epanalipseis)
212     ok1 = target_low <= est1 <= target_high
213     successes_algo1.append(ok1)
214
215     est2 = run_algo2(n_epanalipseis, best_alpha2)
216     ok2 = target_low <= est2 <= target_high
217     successes_algo2.append(ok2)
218
219 if (t + 1) % 50 == 0:
220     print(f" Trial {t+1}: Algo1 running rate={sum(successes_algo1)/(t+1):.3f}, Algo2={sum(successes_algo2)/(t+1):.3f}")
221     sys.stdout.flush()
222
223

```

Αποτελέσματα Monte Carlo (500 trials):

Αλγόριθμος	Περιγραφή	$P(800K \leq \text{εκτίμηση} \leq 1.2M)$
Algo1	1×15-bit, $\alpha=2$	39.8%
Algo2	3×5-bit, $\alpha \approx 2.5$	~41.2%

Συμπεραίνω με την βοήθεια του παραπάνω πινακάκι ότι από το πείραμα ο δεύτερος αλγόριθμος, δηλαδή ο μέσος όρος τριών μικρότερων Morris counters, δίνει καλύτερο ποσοστό επιτυχίας από έναν μόνο μεγάλο Morris counter με $\alpha = 2$. Όμως η βελτίωση δεν είναι πολύ μεγάλη και κανένας από τους δύο αλγορίθμους δεν φτάνει σε ποσοστό επιτυχίας 80%. Άρα ο διαχωρισμός της μνήμης σε περισσότερους counters βοηθά κάπως αλλά δεν μας λύνει πλήρως το πρόβλημα της διακύμανσης.

Άσκηση 2α — Ακριβής Πιθανότητα Επιτυχίας

Ανάλυση κώδικα:

Χρησιμοποιείται η βιβλιοθήκη Fraction της Python για ακριβείς αριθμητικές πράξεις με κλάσματα:

```
4 from fractions import Fraction

11
12 MAX_C = 22 #gia na min megalonei xwris logo to state space
13 MAX_C_TWO = 15
14 N_MAX = 1000
15 c_values = [2, 3, 4, 5]
16
17
18 def next_distribution(dist):
19     #dist[C]= pithanotita na vriskomaste sto C
20     new_dist = {}
21     for C, prob in dist.items():
22         p_increase = Fraction(1, 2**C)
23         p_stay = Fraction(2**C - 1, 2**C)
24         # menw sto idio C
25         new_dist[C] = new_dist.get(C, Fraction(0, 1)) + prob * p_stay
26         #anevenw sto C+1
27         new_C = min(C + 1, MAX_C)
28         new_dist[new_C] = new_dist.get(new_C, Fraction(0, 1)) + prob * p_increase
29
30     return new_dist
31
32 def estimate_from_C(C):
33     return Fraction(2**C-1,1)
34
35 def success_probability(dist, n, c): # gia ayto to n poia h pithanotita i ektimisi na einai anamesa sto n/c kai sto c*n
36     low = Fraction(n, c)
37     high = Fraction(c * n, 1)
38
39     total = Fraction(0, 1) #gia na vrw sto telos tin sinoliki pithanothta
40
41     for C, prob in dist.items():
42         estimate = estimate_from_C(C) # metatrepw c se estimate
43         if low <= estimate <= high:
44             total += prob
45
46     return total
```



```

111 # — A2α —
112 print("=== A2α: Exact probability analysis for c ∈ {2,3,4,5} ===")
113 # arxiki katanomh prin apo inserts eimaste sto C=0
114 dist = {0: Fraction(1, 1)}
115 results = {}
116
117 for c in c_values:
118     results[c] = []
119     # gia kathe n apo 1 ews 1000
120     for n in range(1, N_MAX + 1):
121         dist = next_distribution(dist)
122         for c in c_values:
123             p = success_probability(dist, n, c)
124             results[c].append(p)
125
126 #print ths xeiroters periptwsh gia kathe c
127 for c in c_values:
128     probs = results[c]
129     min_prob = min(probs)
130     worst_n = probs.index(min_prob) + 1
131     print(f"c={c}: min probability={float(min_prob):.4f}, worst n={worst_n}")
132
133
134 plt.figure(figsize=(12, 6))
135 ns = list(range(1, N_MAX + 1))
136
137 for c in c_values:
138     probs = [float(p) for p in results[c]]
139     min_prob = min(results[c])
140     worst_n = results[c].index(min_prob) + 1
141     plt.plot(ns, probs, label=f'c={c} (min={float(min_prob):.4f} at n={worst_n})')
142
143 plt.xlabel('n')
144 plt.ylabel('P(n/c ≤ estimate ≤ c·n)')
145 plt.title('A2α: Success Probability vs n for Morris Counter (α=2)')
146 plt.legend()
147 plt.tight_layout()
148 plt.savefig('plots/A2a_success_prob.png')
149 plt.close()
150 print("Saved plots/A2a_success_prob.png")
151

```

Η `step_distribution` υπολογίζει ακριβώς την κατανομή του C μετά από ένα βήμα `insert`, χρησιμοποιώντας ακριβείς κλασματικές πιθανότητες. Τρέχοντας αυτή για $n=1..1000$, παίρνουμε την ακριβή κατανομή σε κάθε βήμα.

Αποτελέσματα:

c	$\min_{\{1 \leq n \leq 1000\}} P(n/c \leq E_n \leq c \cdot n)$	Χειρότερο n
2	0.4218567224	511
3	0.8049058495	10
4	0.9177510591	509
5	0.9647827148	6

Συμπερασματικά για $c = 2$, η χειρότερη περίπτωση εμφανίζεται στο $n = 511$ με πιθανότητα περίπου 42%. Αυτό συμβαίνει επειδή το $511 = 2^9 - 1$ βρίσκεται πολύ κοντά σε σημείο όπου η εκτίμηση $2^C - 1$ αλλάζει “σκαλοπατί”, οπότε η πιθανότητα να βρεθεί εκτός του διαστήματος $[n/2, 2n]$ είναι μεγαλύτερη.

Όσο μεγαλώνει το c , η πιθανότητα επιτυχίας αυξάνεται σημαντικά, επειδή το αποδεκτό διάστημα $[n/c, cn]$ γίνεται πιο μεγάλο. Για αυτό το $c = 5$ δίνει πολύ καλύτερα αποτελέσματα από το $c = 2$.

Άσκηση 2β — Survival Probability

Ανάλυση κώδικα:


```

47
48 def filter_good_states(dist, n, c):
49     low = Fraction(n, c)
50     high = Fraction(c * n, 1)
51
52     good = {}
53     for C, prob in dist.items():
54         estimate = estimate_from_C(C)
55         if low <= estimate <= high:
56             good[C] = prob
57
58     return good
59
60 results_2b = {}
61 def compute_survival(c, N_MAX=1000):
62     survival = {0: Fraction(1, 1)} #arxika C=0 kai profanws den exw apotixei akoma
63     probs = []
64     for n in range(1, N_MAX + 1):
65         survival = next_distribution(survival)
66         survival = filter_good_states(survival, n, c)
67
68         total_prob = sum(survival.values(), Fraction(0, 1)) #synoliki pithanotita oswn katastasewn eftasan mexri telous
69         probs.append(float(total_prob))
70
71     return probs
72
148 # — A2β —
149 print("\n=== A2β: Probability of staying in bounds for ALL steps 1..1000 ===")
150 results_2b = {}
151 for c in c_values:
152     probs_surv = compute_survival(c, N_MAX=N_MAX)
153     results_2b[c] = probs_surv
154     print(f"c={c}: P(in bounds throughout all {N_MAX} inserts) = {probs_surv[-1]:.10f}")
155

```

Η διαφορά του ερωτήματος αυτού με το A2α είναι ότι εδώ κρατάμε μόνο τις καταστάσεις που ήταν συνεχώς εντός ορίων. Αν σε έστω ένα βήμα η εκτίμηση βγει εκτός ορίων, τότε η κατάσταση αφαιρείται από το results_2b. Η πιθανότητα να παραμείνει η εκτίμηση εντός ορίων για όλα τα 1000 βήματα είναι πολύ μικρότερη από την πιθανότητα του A2α. Αυτό το περιμένω αφού απαιτούνται 1000 ταυτόχρονες επιτυχίες. Ο μετρητής Morris δεν είναι σχεδιασμένος για τόσο συνεχής παρακολούθηση σε κάθε βήμα, αλλά περισσότερο για να δίνει μια καλή συνολική τελική εκτίμηση.

Άσκηση 2γ — Δύο 4-bit Morris Counters, Μέσος Όρος

Ανάλυση κώδικα:

```

13 MAX_C_TWO = 15

```

```

75 def next_joint_dist(joint_dist):
76     new_dist = {}
77     for state in joint_dist:
78         c1, c2 = state
79         prob_state = joint_dist[state]
80
81         # periptwsh 1: den anevainei kanenas
82         p_stay_1 = Fraction(2**c1 - 1, 2**c1)
83         p_stay_2 = Fraction(2**c2 - 1, 2**c2)
84         new_state = (c1, c2)
85         new_prob = prob_state * p_stay_1 * p_stay_2
86         new_dist[new_state] = new_dist.get(new_state, Fraction(0, 1)) + new_prob
87
88         # periptwsh 2: anevainei mono o deyteros
89         p_up_2 = Fraction(1, 2**c2)
90         new_state = (c1, min(c2 + 1, MAX_C))
91         new_prob = prob_state * p_stay_1 * p_up_2
92         new_dist[new_state] = new_dist.get(new_state, Fraction(0, 1)) + new_prob
93
94         # periptwsh 3: anevainei mono o protos
95         p_up_1 = Fraction(1, 2**c1)
96         new_state = (min(c1 + 1, MAX_C), c2)
97         new_prob = prob_state * p_up_1 * p_stay_2
98         new_dist[new_state] = new_dist.get(new_state, Fraction(0, 1)) + new_prob
99
100        # periptwsh 4: anevainoun kai oi 2
101        new_state = (min(c1 + 1, MAX_C), min(c2 + 1, MAX_C))
102        new_prob = prob_state * p_up_1 * p_up_2
103        new_dist[new_state] = new_dist.get(new_state, Fraction(0, 1)) + new_prob
104
105    return new_dist
106 def mean_estimate(c1, c2):
107     est1 = 2**c1 - 1
108     est2 = 2**c2 - 1
109     return Fraction(est1 + est2, 2)
110

```

```

112 def success_prob_joint(joint_dist, n, c):
113     low = Fraction(n, c)
114     high = Fraction(c * n, 1)
115
116     total = Fraction(0, 1)
117
118     for state in joint_dist:
119         c1, c2 = state
120         prob_state = joint_dist[state]
121
122         avg_est = mean_estimate(c1, c2)
123
124         if low <= avg_est <= high:
125             total += prob_state
126
127     return total
128

```

```

179 # — A2γ —
180 print("=== A2γ: Two 4-bit Morris counters ===")
181
182 joint_dist = {(0, 0): Fraction(1, 1)}
183 results_2c = {}
184
185 for c in c_values:
186     results_2c[c] = []
187
188 for n in range(1, N_MAX + 1):
189     joint_dist = next_joint_dist(joint_dist)
190
191     for c in c_values:
192         p = success_prob_joint(joint_dist, n, c)
193         results_2c[c].append(float(p))
194
195 for c in c_values:
196     probs = results_2c[c]
197     min_prob = min(probs)
198     worst_n = probs.index(min_prob) + 1
199     print(f"c={c}: min probability={min_prob:.4f}, worst n={worst_n}")
200
201 plt.figure(figsize=(12, 6))
202 ns = list(range(1, N_MAX + 1))
203
204 for c in c_values:
205     probs = results_2c[c]
206     min_prob = min(probs)
207     worst_n = probs.index(min_prob) + 1
208     plt.plot(ns, probs, label=f'c={c} (min={min_prob:.4f} at n={worst_n})')
209
210 plt.xlabel("n")
211 plt.ylabel("P(n/c <= mean estimate <= c*n)")
212 plt.title("A2γ: Two 4-bit Morris Counters")
213 plt.legend()
214 plt.tight_layout()
215 plt.savefig("plots/A2g_two_4bit.png")
216 plt.close()

```

Στο ερώτημα αυτό χρησιμοποιούνται δύο ανεξάρτητοι 4-bit Morris counters, άρα κάθε μετρητής μπορεί να πάρει τιμές από 0 έως 15. Επομένως, η κοινή κατάσταση του συστήματος περιγράφεται από ένα ζεύγος (C1, C2), οπότε υπάρχουν συνολικά $16 \times 16 = 256$ πιθανές καταστάσεις. Η συνάρτηση `step_joint_distribution` υπολογίζει την ακριβή κατανομή αυτών των ζευγών μετά από κάθε βήμα. Σε κάθε `insert`, ο πρώτος και ο δεύτερος counter μπορούν είτε να μείνουν ίδιοι είτε να αυξηθούν κατά 1, και επειδή θεωρούνται ανεξάρτητοι, η πιθανότητα κάθε κοινής μετάβασης είναι το γινόμενο των δύο αυτών επιμέρους πιθανοτήτων. Στη συνέχεια, η `success_prob_joint` υπολογίζει την πιθανότητα ο μέσος όρος των δύο εκτιμήσεων να βρίσκεται μέσα στο διάστημα $[n/c, c \cdot n]$. Ο μέσος όρος δίνεται από τον αυτόν τον τύπο : $((2^{C1} - 1) + (2^{C2} - 1)) / 2$. Έτσι για κάθε $n \leq 1000$, οι δύο 4-bit counters λειτουργούν ικανοποιητικά και ο μέσος όρος τους δίνει καλή πιθανότητα επιτυχίας. Η χρήση δύο counters βοηθά στη μείωση της διακύμανσης σε σχέση με έναν μόνο μικρό counter. Η λύση όμως όπως φαίνεται και από το γράφημα παραμένει προσεγγιστική και η πιθανότητα επιτυχίας εξαρτάται από το c.

Ενότητα Β: Καταμέτρηση Διακεκριμένων Στοιχείων

Στην ενότητα αυτή μελετώ το πρόβλημα της εκτίμησης του F_0 , δηλαδή του πλήθους των διαφορετικών στοιχείων που εμφανίστηκαν στη ροή δεδομένων.

Άσκηση 1α — Αλγόριθμος Flajolet-Martin (FM), Uniform, Trie

Υλοποίηση Trie από μηδέν:

```
21 # -----Trie-----
22 class TrieNode:
23     def __init__(self):
24         self.children = [None, None] # 2 paidia ena gia bit0 kai ena gia bit1
25         self.is_end = False # flag gia na vlepw an teleiwnei enas arithmos ta bits tou dhladh
26
27 class Trie:
28     def __init__(self, bits):
29         self.root = TrieNode() # ftiaxnw tin riza
30         self.bits = bits # apothikevw t mhkos twv bits twv arithmwv edw 20
31         self.count = 0 # counter gia metrima plnthos diaforetikwn stoiceiwn
32
33     def insert(self, x):
34         node = self.root # ksekinaw apo tin riza
35
36         for i in range(self.bits - 1, -1, -1): # pernw ola ta bits apo to pio shmantiko pros to ligotero
37             bits = (x >> i) & 1 # pairnw to i-osto bit tou arithmou x
38
39             if node.children[bits] is None:
40                 node.children[bits] = TrieNode() # an gia ayto to padi den exw bit to ftiaxnw
41                 node = node.children[bits] # paw sto epomeno epipedo tou Trie
42
43             if node.is_end == False: # elegxw gia diples isagoges
44                 node.is_end = True # afou elegksa thn monadikotita twra kanw true to flag oti teliwse edw o apothikeumenos arithmos
45                 self.count += 1 # distinct count +1
46             return True # afou mpika neo stoiceio
47         return False # an yparxei o arithmos idi mesa den ayksanw to count
48
```

Το Trie αποθηκεύει κάθε 20-bit αριθμό ως μονοπάτι από τη ρίζα ως φύλλο. Κάθε εισαγωγή χρειάζεται $O(d)$ βήματα και εδώ, επειδή το $d = 20$ άρα το κόστος θα είναι σταθερό. Το Trie χρησιμοποιείται για να υπολογίζεται το πραγματικό πλήθος στοιχείων ώστε να γίνεται σύγκριση με την εκτίμηση του FM.

Για τον Αλγόριθμο FM η συνάρτηση `trailing_zeros(x)` υπολογίζει πόσα συνεχόμενα μηδενικά έχει ο αριθμός x στο τέλος της δυαδικής του αναπαράστασης. Ο αλγόριθμος FM κρατά μόνο τη μέγιστη τιμή R που έχει εμφανιστεί μέχρι στιγμής και επιστρέφει ως εκτίμηση το 2^R . Επομένως αν υπάρχουν F_0 διακεκριμένα στοιχεία, τότε αναμένουμε το μέγιστο πλήθος trailing zeros να είναι περίπου $\log_2(F_0)$, άρα η ποσότητα 2^R προσεγγίζει το F_0 . Έτσι η εκτίμηση παίρνει μόνο τιμές που είναι δυνάμεις του 2.

```
11
12 def trailing_zeros(x): #Gia na parw kathe distinct stoiceio kai koitaw posa mhdenika exei sto telos tis diadikhs anaparastashs.
13     if x == 0: # gia tin periptwsh dhadh to 0
14         return 20 # afou douleuvw me 20bitous arithmous
15     count = 0
16     while (x & 1) == 0: # elegxw to teleutaio bit
17         count += 1
18         x >>= 1 # gia deksia metatopisi. afou vrika 0 paw sthn dipla thesi
19     return count
20
```

```

49 # — Bia —
50 # ----- Bia: FM + Trie -----
51
52 print("=== Bia: Flajolet-Martin on uniform distribution ===")
53
54 d = 20
55 n = 1_000_000
56
57 trie = Trie(d) # ftiaxnw Trie gia na krataw to pragmatiko distinct count
58 R = 0 # o FM krataei to megisto trailing zeros mexri twra
59
60 x_points = []
61 real_values = []
62 fm_values = []
63
64 for i in range(1, n + 1):
65     # ftiaxnw ena tuxaio 20-bitito arithmo
66     x = random.randint(0, 2**d - 1)
67
68     # to vazw sto trie gia na exw to pragmatiko distinct count
69     trie.insert(x)
70
71     # ypologizw ta trailing zeros
72     r = trailing_zeros(x)
73
74     # o FM krata to megisto R
75     if r > R:
76         R = r
77
78     # FM estimate
79     estimate = 2 ** R
80
81     # pragmatiko distinct count
82     real_distinct = trie.count
83
84     # den krataw ola ta simeia, mono kathe 10.000
85     if i % 10000 == 0:
86         x_points.append(i)
87         real_values.append(real_distinct)
88         fm_values.append(estimate)
89         print(i, real_distinct, estimate)
90
91
92     R = r
93
94     # FM estimate
95     estimate = 2 ** R
96
97     # pragmatiko distinct count
98     real_distinct = trie.count
99
100     # den krataw ola ta simeia, mono kathe 10.000
101     if i % 10000 == 0:
102         x_points.append(i)
103         real_values.append(real_distinct)
104         fm_values.append(estimate)
105         print(i, real_distinct, estimate)
106
107 # plot
108 plt.figure(figsize=(12, 5))
109 plt.plot(x_points, real_values, label="True distinct", alpha=0.8)
110 plt.step(x_points, fm_values, where="post", label="FM Estimate (2^R)", alpha=0.8)
111
112 plt.xlabel("n (inserts)")
113 plt.ylabel("Distinct count")
114 plt.title("Bia: Flajolet-Martin on Uniform Distribution")
115 plt.legend()
116 plt.tight_layout()
117 plt.savefig("plots/Bia_distinct.png")
118 plt.close()
119
120 print("Saved plots/Bia_distinct.png")

```

Καταλήγω στο εξής συμπέρασμα στο ότι η uniform κατανομή ο FM συμπεριφέρεται αναμενόμενα και δίνει εκτίμηση του πλήθους στοιχείων. Το βασικό του πλεονέκτημα είναι ότι χρειάζεται πολύ μικρή μνήμη, αφού αποθηκεύει μόνο τη μεταβλητή R. Αντιθέτως η εκτίμηση 2^R δεν είναι πολύ ακριβής μιας και μπορεί να διαφέρει από την πραγματική τιμή, αφού αλλάζει μόνο σε δυνάμεις του 2.

Άσκηση 1β — Non-Uniform Κατανομή: Πρόβλημα και Λύση

Ανάλυση κώδικα — γεννήτρια non-uniform:

```

108
109 def generate_nonuniform():
110     x = 0
111
112     for i in range(5): # bits 1-5 (MSB)
113         if random.random() < 0.5: # ta prwta bits exoun pithanothta 0.5 na ginoun 1
114             x |= (1 << (19 - i)) # vazw to 19-i iso me 1
115
116     for i in range(5): # bits 6-10
117         if random.random() < 0.25: # ta prwta bits exoun pithanothta 0.25 na ginoun 1
118             x |= (1 << (14 - i)) # vazw to 14-i iso me 1
119
120     for i in range(5): # bits 11-15
121         if random.random() < 0.125: # ta prwta bits exoun pithanothta 0.125 na ginoun 1
122             x |= (1 << (9 - i)) # vazw to 9-i iso me 1
123
124     for i in range(5): # bits 16-20 (LSB)
125         if random.random() < 0.0625: # ta prwta bits exoun pithanothta 0.0625 na ginoun 1
126             x |= (1 << (4 - i)) # vazw to 4-i iso me 1
127
128     return x
129

```

Τα λιγότερο σημαντικά bits έχουν μικρότερη πιθανότητα να είναι 1, οπότε πολλά από τα παραγόμενα x τελειώνουν σε αρκετά μηδενικά. Αυτό σημαίνει ότι η τιμή των trailing zeros είναι συχνά μεγάλη, όχι επειδή υπάρχουν πραγματικά περισσότερα distinct στοιχεία, αλλά επειδή η ίδια η κατανομή είναι biased. Έτσι ο FM κρατά μεγαλύτερη τιμή R και υπερεκτιμά το πλήθος.

Λύση — χωρίς Hash function:

```

130 print("--- Part 1: Without hash ---")
131
132 R_nohash = 0 # megisto trailing zeros
133 trie_nohash = Trie(d) # gia pragmatiko distinct count
134
135 x_values_nohash = []
136 real_values_nohash = []
137 fm_values_nohash = []
138
139 for i in range(1, n + 1):
140     x = generate_nonuniform()
141
142     trie_nohash.insert(x)
143
144     r = trailing_zeros(x) # gia na vrw trailing zeros panw sto idio to x dhladh h(x)=x
145     if r > R_nohash:
146         R_nohash = r
147
148     estimate = 2 ** R_nohash
149     real_distinct = trie_nohash.count
150
151     if i % 10000 == 0: # kathe 10k vimata krataw shmeia gia to plot
152         x_values_nohash.append(i)
153         real_values_nohash.append(real_distinct)
154         fm_values_nohash.append(estimate)
155         print(i, real_distinct, estimate)
156
157 plt.figure(figsize=(12, 5))
158 plt.plot(x_values_nohash, real_values_nohash, label="True distinct", alpha=0.8)
159 plt.step(x_values_nohash, fm_values_nohash, where="post", label="FM Estimate (no hash)", alpha=0.8)
160 plt.xlabel("n (inserts)")
161 plt.ylabel("Distinct count")
162 plt.title("B1β Part 1: FM on Non-Uniform Distribution (no hash)")
163 plt.legend()
164 plt.tight_layout()
165 plt.savefig("plots/B1b_nonuniform.png")
166 plt.close()
167
168 print("Saved plots/B1b_nonuniform.png")
169

```

Λύση — Hash function:

```
172 print("\n--- Part 2: With hash ---")
173
174 p = 1048583 # arithmos > tou 2**20
175 a = random.randint(1, p - 1)
176 b = random.randint(0, p - 1)
177
178 def my_hash(x):
179     return ((a * x + b) % p) % (2 ** d)
180
181 R_hash = 0
182 trie_hash = Trie(d)
183
184 x_values_hash = []
185 real_values_hash = []
186 fm_values_hash = []
187
188 for i in range(1, n + 1):
189     x = generate_nonuniform()
190
191     trie_hash.insert(x)
192
193     hx = my_hash(x)
194     r = trailing_zeros(hx)
195
196     if r > R_hash:
197         R_hash = r
198
199     estimate = 2 ** R_hash
200     real_distinct = trie_hash.count
201
202     if i % 10000 == 0:
203         x_values_hash.append(i)
204         real_values_hash.append(real_distinct)
205         fm_values_hash.append(estimate)
206         print(i, real_distinct, estimate)
207
208 plt.figure(figsize=(12, 5))
209 plt.plot(x_values_hash, real_values_hash, label="True distinct", alpha=0.8)
210 plt.step(x_values_hash, fm_values_hash, where="post", label="FM Estimate (with hash)", alpha=0.8)
211 plt.xlabel("n (inserts)")
212 plt.ylabel("Distinct count")
213 plt.title("B1β Part 2: FM on Non-Uniform Distribution (with hash)")
214 plt.legend()
```

Η γραμμική hash συνάρτηση $h(x) = ((ax + b) \bmod p) \bmod 2^d$ ανακατεύει τα bits του x . Έτσι παρόλο που τα αρχικά δεδομένα είναι non-uniform, οι hashed τιμές κατανέμονται πολύ πιο κοντά στην ομοιόμορφη κατανομή. Άρα ο αριθμός των trailing zeros δεν επηρεάζεται τόσο από το bias της αρχικής κατανομής και ο FM δίνει πολύ καλύτερη εκτίμηση.

Συμπερασματικά χωρίς hash, ο FM δίνει πολύ κακή εκτίμηση σε non-uniform δεδομένα και συνήθως υπερεκτιμά έντονα το πραγματικό πλήθος. Με τη χρήση hash αποφεύγεται η καταστροφική συμπεριφορά του $h(x)=x$ και η εκτίμηση γίνεται πιο ελεγχόμενη, αλλά ο απλός FM παραμένει χοντρική προσέγγιση.

Άσκηση 2α — Εμπειρικός Έλεγχος Ιδιοτήτων $H(2^d)$

Ανάλυση κώδικα:

```
12 d = 20
13 p = 1048583 # arithmos ligo > apo 2**20
14 n_samples=20000 #Oso megalitero toso pio stathero to observed apotelesma
15
16 def trailing_zeros(x):
17     if x == 0:
18         return d
19     count = 0
20     while (x & 1) == 0:
21         count += 1
22         x >>= 1
23     return count
24
25 def h_hash(x, alpha, beta):
26     return ((alpha * x + beta) % p) % (2**d)
27
28 # --- B2α ---
29 print("=== B2α: Empirically check H(2^d) properties ===")
30 r_values_test = [1, 2, 3, 4, 5]
31 x_values = [0, 1, 42, 1000, 2**10, 2**15, 2**20 - 1]
32 xy_pairs = [(0, 1), (1, 2), (42, 137), (1000, 2000), (2**10, 2**10 + 1)]
33
34 # ----- Property 1 -----
35 print(f"\nFor x = {x}")
36 for x in [0, 1, 42, 1000]: # Prepei thewritina na einai peripou 1/(2^r)
37     for r in [1, 2, 3, 4]:
38         success = 0
39         for i in range(n_samples):
40             alpha = random.randint(1, p - 1)
41             beta = random.randint(0, p - 1)
42
43             hx = h_hash(x, alpha, beta)
44             if trailing_zeros(hx) >= r:
45                 success += 1
46         observed = success / n_samples
47         expected = 1.0 / (2**r)
48         print(f"r={r}: observed={observed:.4f}, expected={expected:.4f}")
49
50 # ----- Property 2 -----
51 print("\nProperty 2: P(tr(h(x)) >= r and tr(h(y)) >= r) ~ 1/4^r")
52
53 # ----- Property 2 -----
54 print(f"\nProperty 2: P(tr(h(x)) >= r and tr(h(y)) >= r) ~ 1/4^r")
55 for (x, y) in [(1, 2), (42, 137), (1000, 2000)]: # Prepei thewritina na einai peripou 1/(4^r)
56     for r in [1, 2, 3]:
57         success = 0
58         for k in range(n_samples):
59             alpha = random.randint(1, p - 1)
60             beta = random.randint(0, p - 1)
61             hx = h_hash(x, alpha, beta)
62             hy = h_hash(y, alpha, beta)
63
64             if trailing_zeros(hx) >= r and trailing_zeros(hy) >= r:
65                 success += 1
66         observed = success / n_samples
67         expected = 1.0 / (4**r)
68         print(f"r={r}: observed={observed:.4f}, expected={expected:.4f}")
69
70 # --- B2β ---
```

Στο ερώτημα αυτό ελέγχω εμπειρικά αν η οικογένεια hash συναρτήσεων

$h(x) = ((\alpha x + \beta) \bmod p) \bmod 2^d$ συμπεριφέρεται όπως χρειάζεται ο αλγόριθμος. Για πολλές τυχαίες επιλογές των α και β , υπολογίζω πειραματικά τις πιθανότητες:

$P(\text{tr}(h(x)) \geq r)$ και

$P(\text{tr}(h(x)) \geq r \text{ και } \text{tr}(h(y)) \geq r)$

και τις συγκρίνω με τις θεωρητικές τιμές $1/2^r$ και $1/4^r$ αντίστοιχα. Οι μικρές αποκλίσεις οφείλονται στο ότι η κατανομή που προκύπτει από το $\bmod p$ και μετά $\bmod 2^d$ δεν είναι απολύτως ομοιόμορφη. Οπότε συμπερασματικά η $H(2^d)$ φαίνεται στην πράξη να λειτουργεί αρκετά καλά, αφού οι μετρούμενες πιθανότητες είναι κοντά στις θεωρητικές τιμές. Αλλά, δεν ικανοποιεί τις ιδιότητες ακριβώς, οπότε για θεωρητικές εγγυήσεις είναι πιο ασφαλής η οικογένεια $V(2^d)$.

Άσκηση 2β — Απόδειξη: $V(2^d)$ Ικανοποιεί τις Ιδιότητες

Δομή της οικογένειας $V(2^d)$

Κάθε συνάρτηση στην $V(2^d)$ ορίζεται από affine μετασχηματισμό πάνω από $GF(2)$. Κάθε bit εξόδου υπολογίζεται ανεξάρτητα:

$$h_i(x) = (\alpha_{\{i,1\}}x_1 + \alpha_{\{i,2\}}x_2 + \dots + \alpha_{\{i,d\}}x_d + \beta_i) \bmod 2$$

Απόδειξη:

Για την Ιδιότητα 1 ισχύει:

$$P(\text{tr}(h(x)) \geq r) = P(h_1(x)=0, h_2(x)=0, \dots, h_r(x)=0).$$

Για κάθε θέση i , το bit $h_i(x)$ είναι ομοιόμορφο με πιθανότητα $1/2$ και τα output bits είναι ανεξάρτητα μεταξύ τους. Άρα:

$$P(\text{tr}(h(x)) \geq r) = (1/2)^r = 1/2^r.$$

Για την Ιδιότητα 2 ισχύει:

$$P(\text{tr}(h(x)) \geq r \text{ και } \text{tr}(h(y)) \geq r)$$

$$= P(h_1(x)=h_1(y)=0, \dots, h_r(x)=h_r(y)=0).$$

Τέλος για κάθε θέση i και για $x \neq y$, ισχύει ότι:

$$P(h_i(x)=0 \text{ και } h_i(y)=0) = 1/4,$$

ενώ οι διαφορετικές θέσεις i είναι ανεξάρτητες. Οπότε:

$$P(\text{tr}(h(x)) \geq r \text{ και } \text{tr}(h(y)) \geq r) = (1/4)^r = 1/4^r.$$

Συμπεραίνω ότι η οικογένεια $V(2^d)$ ικανοποιεί ακριβώς τις δύο ιδιότητες που χρειάζεται ο αλγόριθμος FM. Για αυτό προσφέρει τις σωστές θεωρητικές εγγυήσεις και είναι καλύτερη από θεωρητικής πλευράς.

Άσκηση 2γ — Σύγκριση $V(2^d)$ vs $H(2^d)$

Κριτήριο	$V(2^d)$	$H(2^d)$
Ακρίβεια ιδιοτήτων	Ακριβής (αποδεδειγμένη)	Κατά προσέγγιση
Bits ανά συνάρτηση	$d(d+1)$ bits π.χ 420 ($d=20$)	Περίπου $2d$ bits π.χ ~40 για $d=20$
Κόστος αξιολόγησης	d XOR + d AND	1 mult + mod p + mod 2^d
Πρακτικό $d \leq 20$	Ναι	Ναι
Πρακτικό $d = 100$	Όχι γιατί θέλει 10.000 bits/fn	Ναι

Συμπεραίνοντας καταλήγω στο ότι το $V(2^d)$ είναι θεωρητικά καλύτερο, αλλά κοστίζει πολύ περισσότερο. Το $H(2^d)$ δεν είναι τέλειο θεωρητικά, αλλά είναι πολύ πιο πρακτικό. Για αυτό στην πράξη συνήθως προτιμάται το $H(2^d)$, ειδικά όταν το d μεγαλώνει.

Άσκηση 3α — BJKST Αλγόριθμος

Δομή δεδομένων TopKSmallest:

```
10 # oi statheres tis ekfwnhsis
11 m = 2**20
12 M = m**3
13
14 q = 1048583
15 p = q**3
16
17 def make_hash():
18     a = random.randint(1, p - 1)
19     b = random.randint(0, p - 1)
20
21     def h(x):
22         return ((a * x + b) % p) % M + 1
23
24     return h
25
26
27 class TopKSmallest:
28     def __init__(self, k):
29         self.k = k
30         self.heap = [] # tha krataei arnitikes times gia na to xrisimopoihsoume ws max-heap
31         self.values = set()
32
33     def add(self, v):
34         # den thelουμε duplicates
35         if v in self.values:
36             return
37
38         # an den exei gemisei akoma, bale to
39         if len(self.heap) < self.k:
40             heapq.heappush(self.heap, -v)
41             self.values.add(v)
42             return
43
44         # to megalutero apo ta k mikrotera einai stin korifi
45         current_biggest = -self.heap[0]
46
47         # an to neo v einai mikrotero, tote prepei na mpei mesa
48         if v < current_biggest:
49             removed = -heapq.heapreplace(self.heap, -v)
50             self.values.remove(removed)
51             self.values.add(v)
52
53
54     def kth_smallest(self):
55         if len(self.heap) < self.k:
56             return None
57         return -self.heap[0]
```

Η TopKSmallest διατηρεί τα t μικρότερα διαφορετικά hash values που έχουν εμφανιστεί. Και χρησιμοποιεί max-heap με αρνητικές τιμές έτσι ώστε το μεγαλύτερο από αυτά να βρίσκεται στην κορυφή και να μπορούμε εύκολα να ελέγξουμε αν ένα νέο hash value πρέπει να μπει στη δομή.

Κύρια κλάση BJKST:

```

59 class BJKST:
60     def __init__(self, eps):
61         self.eps = eps
62         self.t = math.ceil(96 / (eps ** 2))
63         self.h = make_hash()
64         self.smallest_hashes = TopKSmallest(self.t)
65
66     def insert(self, x):
67         x=x+1
68         hx = self.h(x)
69         self.smallest_hashes.add(hx)
70
71     def query(self):
72         v = self.smallest_hashes.kth_smallest()
73
74         if v is None:
75             return 0
76
77         return (self.t * M) / v
78
79
80     def generate_uniform(d=20):
81         return random.randint(0, 2**d - 1)
82
83
84     def generate_nonuniform():
85         x = 0
86
87         # bits 1-5
88         for i in range(5):
89             if random.random() < 0.5:
90                 x |= (1 << (19 - i))
91
92         # bits 6-10
93         for i in range(5):
94             if random.random() < 0.25:
95                 x |= (1 << (14 - i))
96
97         # bits 11-15

```

```

97     # bits 11-15
98     for i in range(5):
99         if random.random() < 0.125:
100             x |= (1 << (9 - i))
101
102     # bits 16-20
103     for i in range(5):
104         if random.random() < 0.0625:
105             x |= (1 << (4 - i))
106
107     return x
108
109
110 def run_bjkst_experiment(eps, gen_fn, dist_name, plot_filename, n=1_000_000):
111     print(f"\n=== B3α: BJKST, eps={eps}, distribution={dist_name} ===")
112
113     bjkst = BJKST(eps)
114     real_seen = set()
115
116     x_values = []
117     real_values = []
118     est_values = []
119
120     for i in range(1, n + 1):
121         x = gen_fn()
122         real_seen.add(x)
123         bjkst.insert(x)
124
125         real_distinct = len(real_seen)
126
127         # kratav dedomena gia plot mono otan exw toulaxiston t distinct
128         if i % 50000 == 0 and real_distinct >= bjkst.t:
129             estimate = bjkst.query()
130
131             x_values.append(i)
132             real_values.append(real_distinct)
133             est_values.append(estimate)
134
135             print(i, real_distinct, estimate)
136
137     plt.figure(figsize=(12, 5))
138     plt.plot(x_values, real_values, label="True distinct", alpha=0.8)
139     plt.plot(x_values, est_values, label=f"BJKST estimate (eps={eps})", alpha=0.8)
140
141     # kratav dedomena gia plot mono otan exw toulaxiston t distinct
142     if i % 50000 == 0 and real_distinct >= bjkst.t:
143         estimate = bjkst.query()
144
145         x_values.append(i)
146         real_values.append(real_distinct)
147         est_values.append(estimate)
148
149         print(i, real_distinct, estimate)
150
151     plt.figure(figsize=(12, 5))
152     plt.plot(x_values, real_values, label="True distinct", alpha=0.8)
153     plt.plot(x_values, est_values, label=f"BJKST estimate (eps={eps})", alpha=0.8)
154     plt.xlabel("n (inserts)")
155     plt.ylabel("Distinct count")
156     plt.title(f"B3α: BJKST on {dist_name} distribution")
157     plt.legend()
158     plt.tight_layout()
159     plt.savefig(plot_filename)
160     plt.close()
161
162     print(f"Saved {plot_filename}")
163
164 # paradeigmata runs
165 run_bjkst_experiment(0.1, generate_uniform, "uniform", "plots/B3_uniform_eps01.png")
166 run_bjkst_experiment(0.05, generate_uniform, "uniform", "plots/B3_uniform_eps005.png")
167 run_bjkst_experiment(0.1, generate_nonuniform, "non-uniform", "plots/B3_nonuniform_eps01.png")
168 run_bjkst_experiment(0.05, generate_nonuniform, "non-uniform", "plots/B3_nonuniform_eps005.png")

```

Η εκτίμηση tM/v βασίζεται στην ιδέα ότι αν τα hash values κατανέμονται περίπου ομοιόμορφα στο $[1, M]$, τότε η t-οστή μικρότερη τιμή είναι περίπου $t \cdot M/F_0$. Άρα μπορεί να εκτιμηθεί το πλήθος των distinct στοιχείων με τον τύπο $F_0 \approx tM/v$.

Σύγκριση εγγυήσεων BJKST vs FM

Κριτήριο	FM	BJKST
Ακρίβεια	Πάντα δύναμη 2, σφάλμα έως $2\times$	$(1-\epsilon)F_0 \leq N \leq (1+\epsilon)F_0$

Ρυθμιζόμενο ϵ	Όχι	Ναι — $\epsilon=0.1$ ή 0.05
Πιθανότητα επιτυχίας	Χωρίς εγγύηση	$\geq 2/3 - 1/m$
Non-uniform χωρίς hash	Δίνει κακή εκτίμηση	Η hash βοηθά να δημιουργηθεί σταθερή και ομοιόμορφη εκτίμηση
Χώρος	$O(\log m)$ bits	$O((1/\epsilon^2) \cdot \log m)$ bits

Τα αποτελέσματα για $\epsilon=0.1$ ($t=9600$) και $\epsilon=0.05$ ($t=38400$) μας δείχνουν ότι ο BJKST παρακολουθεί με ακρίβεια το πραγματικό F_0 και στις δύο κατανομές. Οπότε μικρότερο ϵ ισούται με καλύτερη ακρίβεια αλλά περισσότερη μνήμη.

Συμπερασματικά ο BJKST είναι σαφώς καλύτερος του FM αφού δίνει μια πιο αυστηρή εγγύηση ϵ -ακρίβειας, επεξεργάζεται οποιαδήποτε κατανομή μέσω hash, και η παράμετρος ϵ επιτρέπει trade-off μεταξύ ακρίβειας και μνήμης.

Άσκηση 3β — Median Trick: $P \geq 99.9\%$

Ανάλυση κώδικα:

```

157 # — B3β —
158 print("\n=== B3β: Median trick for success probability ≥ 99.9% ===")
159
160 delta = 0.001
161 K_real = 18 * math.log(1 / delta)
162 K = math.ceil(K_real)
163
164 print("delta =", delta)
165 print("K real =", K_real)
166 print("K =", K)
167 print("Failure bound =", math.exp(-K / 18))
168 print("Success probability >=", 1 - math.exp(-K / 18))
169
170
171 def median_trick_estimate(eps, gen_fn, n=1_000_000):
172     estimates = []
173
174     for _ in range(K):
175         bjkst = BJKST(eps)
176         for _ in range(n):
177             x = gen_fn()
178             bjkst.insert(x)
179             estimates.append(bjkst.query())
180
181     estimates.sort()
182     return estimates[len(estimates) // 2]
183
184
185 print("\nMedian trick example:")
186 est_med = median_trick_estimate(0.1, generate_uniform, n=200000)
187 print("Median estimate for eps=0.1 on uniform data =", est_med)

```

Για κάθε αντίγραφο του BJKST, η πιθανότητα επιτυχίας είναι τουλάχιστον $2/3$. Αν τρέξουμε K ανεξάρτητα αντίγραφα και πάρουμε τη διάμεσο των εκτιμήσεων, τότε αποτυχία θα έχουμε μόνο αν αποτύχουν περισσότερα από τα μισά αντίγραφα. Με την χρήση του Hoeffding bound προκύπτει ότι: $P(\text{αποτυχίας}) \leq \exp(-K/18)$

Για πιθανότητα αποτυχίας το πολύ $\delta = 0.001$, αρκεί: $\exp(-K/18) \leq 0.001$

οπότε: $K \geq 18 \ln(1000) \approx 124.3$

Άρα θα επιλέξουμε $K = 125$, ώστε να εξασφαλίζεται η πιθανότητα επιτυχίας τουλάχιστον 99.9% .

Συμπερασματικά ο median trick επιτρέπει να αυξήσουμε την πιθανότητα επιτυχίας του BJKST σε επίπεδο $1-\delta$, χρησιμοποιώντας $K = O(\log(1/\delta))$ ανεξάρτητα αντίγραφα. Για $\delta = 0.001$ αρκούν 125 αντίγραφα. Το κόστος είναι ότι η συνολική μνήμη αυξάνεται ανάλογα με τον αριθμό των αντιγράφων αφού εδώ γίνεται περίπου 125 φορές μεγαλύτερη από ένα μόνο αντίγραφο.

Ενότητα Γ: Φίλτρα Bloom

Τα φίλτρα Bloom είναι πιθανοτικές δομές δεδομένων που χρησιμοποιούνται για να ελέγξουμε αν ένα στοιχείο ανήκει σε ένα σύνολο, χρησιμοποιώντας πολύ μικρό χώρο μνήμης. Το βασικό τους χαρακτηριστικό είναι ότι δεν έχουν false negatives, αλλά μπορεί να εμφανίσουν ελεγχόμενα false positives. Για αυτό χρησιμοποιούνται συχνά σε βάσεις δεδομένων, δίκτυα και κατανεμημένα συστήματα.

Άσκηση 1α — Υπολογισμός Τομής με Bloom Filters

Hash function για 100-bit αρχεία:

```
1 import random
2 import matplotlib.pyplot as plt
3
4 random.seed(42)
5
6 # ----- Bloom Filter -----
7 p = 1048583
8
9 def split_100bit_string(s):
10     parts = []
11     for i in range(0, 100, 20):
12         parts.append(int(s[i:i+20], 2))
13     return parts
14
15 def make_bloom_hash():
16     a = random.randint(1, p - 1)
17     b = random.randint(0, p - 1)
18
19     def h(s):
20         c = 0
21         for part in split_100bit_string(s):
22             c = (a * (part + c) + b) % p
23         return c
24
25     return h
26
27
28
29 class BloomFilter:
30     def __init__(self, m, k):
31         self.m = m
32         self.k = k
33         self.bits = [0] * m
34         self.hashes = [make_bloom_hash() for _ in range(k)]
35
36     def positions(self, item):
37         pos = []
38         for h in self.hashes:
39             pos.append(h(item) * self.m)
40         return pos
41
42     def add(self, item):
43         for p in self.positions(item):
44             self.bits[p] = 1
```

Η hash επεξεργάζεται αλυσιδωτά τα 5 κομμάτια του αρχείου. Κάθε κομμάτι «ανακατεύεται» με το αποτέλεσμα του προηγούμενου, δίνοντας καλή διασπορά για οποιαδήποτε μορφή αρχείου.

```

28
29 class BloomFilter:
30     def __init__(self, m, k):
31         self.m = m
32         self.k = k
33         self.bits = [0] * m
34         self.hashes = [make_bloom_hash() for _ in range(k)]
35
36     def positions(self, item):
37         pos = []
38         for h in self.hashes:
39             pos.append(h(item) % self.m)
40         return pos
41
42     def add(self, item):
43         for p in self.positions(item):
44             self.bits[p] = 1
45
46     def contains(self, item):
47         for p in self.positions(item):
48             if self.bits[p] == 0:
49                 return False
50         return True
51
52 # ----- Dimiourgia arxeiwn -----
53
54 def make_random_file():
55     # 100-bit arxeio ws string
56     s = ""
57     for _ in range(100):
58         s += random.choice(["0", "1"])
59     return s
60
61 def make_data():
62     common = set()
63
64     while len(common) < 10:
65         common.add(make_random_file())
66
67     A = set(common)
68     B = set(common)
69
70     while len(A) < 100000:

```

```

52
53 def make_random_file():
54     # 100-bit arxeio ws string
55     s = ""
56     for _ in range(100):
57         s += random.choice(["0", "1"])
58     return s
59
60
61 def make_data():
62     common = set()
63
64     while len(common) < 10:
65         common.add(make_random_file())
66
67     A = set(common)
68     B = set(common)
69
70     while len(A) < 100000:
71         x = make_random_file()
72         if x not in B:
73             A.add(x)
74
75     while len(B) < 100000:
76         x = make_random_file()
77         if x not in A:
78             B.add(x)
79
80     return A, B, common
81
82
83 # ----- Ena gyros protokollou -----
84
85 def one_round(sender_set, receiver_candidates, m, k):
86     bf = BloomFilter(m, k)
87
88     for item in sender_set:
89         bf.add(item)
90
91     new_candidates = set()
92
93     for item in receiver_candidates:
94         if bf.contains(item):

```

```

85 def one_round(sender_set, receiver_candidates, m, k):
86     bf = BloomFilter(m, k)
87
88     for item in sender_set:
89         bf.add(item)
90
91     new_candidates = set()
92
93     for item in receiver_candidates:
94         if bf.contains(item):
95             new_candidates.add(item)
96
97     communication = m # steilame m bits tou bloom filter
98     return new_candidates, communication
99
100
101 # ----- Protokollo pollwn gyrwn -----
102
103 def run_protocol(A, B, rounds, k, mode="fixed"):
104     candidates_A = set(A)
105     candidates_B = set(B)
106
107     total_communication = 0
108     last_B_positives = set(B)
109
110     for r in range(rounds):
111         if r % 2 == 0:
112             # A -> B
113             if mode == "fixed":
114                 m = 500000
115             else:
116                 m = max(10, 5 * len(candidates_A))
117
118             candidates_B, comm = one_round(candidates_A, candidates_B, m, k)
119             total_communication += comm
120
121             # kratame auto pou vrike o B sto teleutaio filter tou A
122             last_B_positives = set(candidates_B)
123
124         else:
125             # B -> A
126             if mode == "fixed":
127                 m = 500000
128             else:
129                 m = max(10, 5 * len(candidates_B))
130
131             candidates_A, comm = one_round(candidates_B, candidates_A, m, k)
132             total_communication += comm
133
134     count = len(last_B_positives)
135     return last_B_positives, count, total_communication
136
137
138 # ----- Peirama -----
139
140 A, B, common = make_data()
141
142 results = []
143
144 for mode in ["fixed", "dynamic"]:
145     for rounds in [1, 2, 3, 4, 5]:
146         for k in [1, 2, 3, 5, 10]:
147             last_B_positives, count, total_comm = run_protocol(A, B, rounds, k, mode)
148
149             all_common_found = common.issubset(last_B_positives)
150
151             results.append((mode, rounds, k, count, total_comm, all_common_found))
152
153             print("mode =", mode,
154                   "rounds =", rounds,
155                   "k =", k,
156                   "count =", count,
157                   "communication =", total_comm,
158                   "all common found =", all_common_found)
159
160
161 # ----- Plot -----
162 xs = []
163 ys = []
164
165 for mode, rounds, k, count, total_comm, ok in results:
166     if ok:
167         xs.append(f"{mode[:3]}-{rounds}r-{k}k")
168         ys.append(count)
169

```

```

156         "count =", count,
157         "communication =", total_comm,
158         "all common found =", all_common_found)
159
160
161     # ----- Plot -----
162     xs = []
163     ys = []
164
165     for mode, rounds, k, count, total_comm, ok in results:
166         if ok:
167             xs.append(f"{mode[:3]}-{rounds}r-{k}k")
168             ys.append(count)
169
170     plt.figure(figsize=(14, 6))
171     plt.bar(xs, ys)
172     plt.xticks(rotation=60)
173     plt.xlabel("setting")
174     plt.ylabel("count")
175     plt.title("Fla: Bloom filter intersection (count for B on last filter from A)")
176     plt.tight_layout()
177     plt.savefig("plots/Gla_bloom_intersection.png")
178     plt.close()
179
180     print("Saved plots/Gla_bloom_intersection.png")
181
182

```

Στο ερώτημα αυτό έκανα ένα πρωτόκολλο για να βρώ κοινά αρχεία ανάμεσα σε δύο πολύ μεγάλα σύνολα, χρησιμοποιώντας Bloom filters. Δημιουργώ δύο σύνολα Α και Β με 100.000 αρχεία 100 bits το καθένα, έτσι ώστε να υπάρχουν ακριβώς 10 κοινά αρχεία. Για κάθε αρχείο εφαρμόζω τη συνάρτηση κατακερματισμού που δίνεται στην εκφώνηση. Πιο συγκεκριμένα, το αρχείο χωρίζεται σε 5 κομμάτια των 20 bits, τα οποία μετατρέπονται σε αριθμούς, και στη συνέχεια από αυτά υπολογίζεται η τελική hash τιμή modulo το μέγεθος του Bloom filter. Σε κάθε γύρο επικοινωνίας ο ένας server στέλνει στον άλλον ένα Bloom filter που έχει κατασκευάσει από το τρέχον σύνολο υποψηφίων στοιχείων. Ο παραλήπτης ελέγχει ποια από τα δικά του στοιχεία περνούν το φίλτρο και κρατά μόνο αυτά ως νέα υποψήφια. Με αυτόν τον τρόπο, σε κάθε γύρο προσπαθώ να μειώσω τα false positives και να μείνουμε πιο κοντά στα πραγματικά κοινά αρχεία. Θα δοκιμάσω διαφορετικές τιμές για το πλήθος των hash functions k και για το πλήθος των γύρων επικοινωνίας. Για κάθε συνδυασμό μετρήσαμε το count, δηλαδή πόσα αρχεία βρήκε θετικά ο Β στο τελευταίο Bloom filter που του έστειλε ο Α. Επίσης ελέγξω αν μέσα σε αυτά παραμένουν και τα 10 πραγματικά κοινά αρχεία. Στόχος είναι το count να είναι όσο γίνεται μικρότερο, χωρίς όμως να χαθούν τα κοινά στοιχεία. Άρα καλύτερος συνδυασμός θεωρείται αυτός που κρατά και τα 10 κοινά αρχεία, ενώ παράλληλα μειώνει όσο γίνεται τα περιττά positives.

Άσκηση 1β — Μαθηματική Εξήγηση Βελτίωσης με Γύρους

Σε κάθε γύρο του πρωτοκόλλου, η πλευρά που κρατά τους τρέχοντες υποψηφίους κατασκευάζει νέο Bloom filter μόνο για αυτά τα αρχεία. Η άλλη πλευρά χρησιμοποιεί το φίλτρο για να απορρίψει όσα αρχεία είναι σίγουρα εκτός τομής και να κρατήσει μόνο όσα περνούν τον έλεγχο. Με αυτόν τον τρόπο, το πλήθος των υποψηφίων συνήθως μειώνεται από γύρο σε γύρο. Τα πραγματικά κοινά αρχεία παραμένουν ενώ ένα μέρος από τα false positives απομακρύνεται. Έτσι μετά από περισσότερους γύρους, το τελικό candidate set πλησιάζει περισσότερο το πραγματικό μέγεθος της τομής τα 10 κοινά αρχεία. Οπότε η χρήση πολλών γύρων βοηθά στη μείωση των false positives, επειδή το φιλτράρισμα επαναλαμβάνεται πάνω σε όλο και μικρότερα σύνολα υποψηφίων. Έτσι η ποιότητα του αποτελέσματος συνήθως βελτιώνεται όσο αυξάνονται οι γύροι αν και το τελικό όφελος πρέπει να αξιολογείται μαζί με το κόστος επικοινωνίας.

Άσκηση 2α — Packet Tracing με Bloom Filters

Για κάθε router κατασκευάζεται ένα Bloom filter 10.000 bits και επιλέγονται οι hash functions του. Στη συνέχεια δημιουργούνται 100.000 τυχαία μηνύματα, όπου κάθε μήνυμα ξεκινά από έναν τυχαίο source router (1..10) και ακολουθεί τυχαία μια διαδρομή μέχρι τον terminal router n . Κάθε router από τον οποίο περνά το μήνυμα το καταγράφει στο Bloom filter του. Για την ιχνηλάτηση, ο αλγόριθμος ξεκινά από τον terminal router και προχωρά προς τα πίσω χρησιμοποιώντας το γράφημα `adj_in`. Εξετάζονται μόνο οι routers των οποίων το Bloom filter απαντά ΝΑΙ για το συγκεκριμένο μήνυμα. Η αναζήτηση υλοποιείται με iterative DFS using stack. Αν στο τέλος μόνο ο πραγματικός source router αναγνωρίζεται ως πιθανή πηγή, τότε η ιχνηλάτηση μετρίεται ως επιτυχής.

```
9  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
10  GRAPHS_DIR = BASE_DIR
11  PLOTS_DIR = os.path.join(BASE_DIR, "plots")
12  os.makedirs(PLOTS_DIR, exist_ok=True)
13
14  BLOOM_SIZE = 10000
15  NUM_MESSAGES = 100000
16  NUM_SOURCE_ROUTERS = 10
17  p = 1048583 # πρώτος λίγο πάνω από 2^20
18
19
20  # -----voithitikes synarthseis-----
21
22  def split_100bit(x):
23      parts = []
24      for _ in range(5):
25          parts.append(x & ((1 << 20) - 1))
26          x >>= 20
27      return parts[::-1]
28
29
30  def make_hash(alpha, beta, N):
31      def h(x):
32          c = 0
33          for part in split_100bit(x):
34              c = (alpha * (part + c) + beta) % p
35          return c % N
36      return h
37
38
39  def load_graph(filename):
40      with open(filename) as f:
41          n, m = map(int, f.readline().split())
42
43          adj_out = {i: [] for i in range(1, n + 1)}
```

```

43     adj_out = {i: [] for i in range(1, n + 1)}
44     adj_in = {i: [] for i in range(1, n + 1)}
45
46     for _ in range(m):
47         x, y = map(int, f.readline().split())
48         adj_out[x].append(y)
49         adj_in[y].append(x)
50
51     return n, adj_out, adj_in
52
53
54     # -----main prosomoiosh-----
55
56     def simulate_and_trace(n, adj_out, adj_in, k_hash):
57         # bloom filters για κάθε router
58         bloom_filters = [bytearray((BLOOM_SIZE + 7) // 8) for _ in range(n + 1)]
59
60         # οι hash functions για router
61         router_hashes = [[] for _ in range(n + 1)]
62         for router in range(1, n + 1):
63             for _ in range(k_hash):
64                 a = random.randint(1, p - 1)
65                 b = random.randint(0, p - 1)
66                 router_hashes[router].append(make_hash(a, b, BLOOM_SIZE))
67
68         # insert στο bloom filter
69         def bf_add(router, msg):
70             for h in router_hashes[router]:
71                 pos = h(msg)
72                 byte_index = pos >> 3
73                 bit_index = pos & 7
74                 bloom_filters[router][byte_index] |= (1 << bit_index)
75
76         # lookup στο bloom filter
77         def bf_check(router, msg):
78             for h in router_hashes[router]:
79                 pos = h(msg)
80                 byte_index = pos >> 3
81                 bit_index = pos & 7
82                 if not (bloom_filters[router][byte_index] & (1 << bit_index)):
83                     return False
84             return True
85

```

```

85
86     messages = []
87     true_sources = []
88
89     for _ in range(NUM_MESSAGES):
90         msg = random.randint(0, 2**100 - 1)
91         source = random.randint(1, NUM_SOURCE_ROUTERS)
92
93         messages.append(msg)
94         true_sources.append(source)
95
96         current = source
97         bf_add(current, msg)
98
99         visited_on_path = {current}
100
101         while current != n:
102             next_nodes = adj_out[current]
103
104             if not next_nodes:
105                 break
106
107             # αποφεύγω κύκλους όσο γίνεται
108             choices = [v for v in next_nodes if v not in visited_on_path]
109             if not choices:
110                 choices = next_nodes
111
112             current = random.choice(choices)
113             visited_on_path.add(current)
114             bf_add(current, msg)
115
116     successful = 0
117
118     for msg, real_source in zip(messages, true_sources):
119         stack = [n]
120         visited = set()
121         claimed_sources = set()
122
123         while stack:
124             router = stack.pop()
125
126             if router in visited:
127                 continue

```

```

127         continue
128         visited.add(router)
129
130         if not bf_check(router, msg):
131             continue
132
133         # or is it source router
134         if 1 <= router <= NUM_SOURCE_ROUTERS:
135             claimed_sources.add(router)
136         else:
137             for pred in adj_in[router]:
138                 if pred not in visited:
139                     stack.append(pred)
140
141         if claimed_sources == {real_source}:
142             successful += 1
143
144     return successful
145
146 # -----main-----
147
148 hash_func_options = [1, 2, 3, 5, 7, 10]
149
150 graph_files = sorted(
151     f for f in os.listdir(GRAPHS_DIR)
152     if f.startswith("graph") and f.endswith(".txt")
153 )
154
155 results_by_graph = {}
156
157 for gf in graph_files:
158     graph_name = gf.replace(".txt", "")
159     path = os.path.join(GRAPHS_DIR, gf)
160
161     n, adj_out, adj_in = load_graph(path)
162
163     print("\n" + "=" * 50)
164     print("Graph:", graph_name)
165     print("k_hash    successful    rate")
166
167     results_by_graph[graph_name] = {}
168
169     for k_hash in hash_func_options:
170         random.seed(42)
171         succ = simulate_and_trace(n, adj_out, adj_in, k_hash)
172         rate = succ / NUM_MESSAGES
173
174         print(k_hash, succ, rate)
175
176         results_by_graph[graph_name][k_hash] = succ
177
178 # -----plot-----
179
180 n_graphs = len(results_by_graph)
181 fig, axes = plt.subplots(1, n_graphs, figsize=(4 * n_graphs, 5))
182
183 if n_graphs == 1:
184     axes = [axes]
185
186 for ax, (name, res) in zip(axes, results_by_graph.items()):
187     ks = sorted(res.keys())
188     rates = [res[k] / NUM_MESSAGES * 100 for k in ks]
189
190     ax.plot(ks, rates, marker="o")
191     ax.set_title(name)
192     ax.set_xlabel("Number of hash functions")
193     ax.set_ylabel("Successful trace rate (%)")
194     ax.set_ylim(0, 105)
195     ax.grid(True, alpha=0.4)
196
197 plt.tight_layout()
198 plot_path = os.path.join(PLOTS_DIR, "D_packet_tracing.png")
199 plt.savefig(plot_path)
200 plt.close()
201
202 print("\nPlot saved to", plot_path)
203
204

```

Γιατί όμως κάποια routers δέχονται πολλά requests? Διότι σε γράφους με κόμβους μεγάλου in-degree, πολλά μονοπάτια συγκλίνουν στους ίδιους routers. Κατά την αναστροφή ιχνηλάτηση, αν ένας τέτοιος router δώσει false positive, τότε η αναζήτηση

εξαπλώνεται και προς πολλούς predecessors του. Έτσι αυξάνονται τα requests και η ιχνηλάτηση γίνεται δυσκολότερη.

Αποτελέσματα ανά γράφο

Γράφος	Δομή	Βέλτιστο k	Επίδοση
graphA1/A2	Παράλληλες αλυσίδες	από το run	Καλή επίδοση μόνο για μικρό k
graphB1/B2	Σύγκλιση σε στάδιο	από το run	Χαμηλή έως μέτρια επίδοση
graphC1/C2/C3	Πλήρης bipartite	από το run	Εξαρτάται πολύ από τον γράφο, με τον C3 να ξεχωρίζει για μικρό k
graphD	Δένδρο	από το run	Από τις καλύτερες επιδόσεις συνολικά

Άσκηση 2β — Διαφορά Επίδοσης Μεταξύ Γράφων

Ο βέλτιστος αριθμός hash functions επηρεάζει άμεσα το false positive rate των Bloom filters. Αν είναι πολύ μικρός, το false positive rate παραμένει υψηλό. Αν είναι πολύ μεγάλος, τα φίλτρα γεμίζουν πιο γρήγορα και πάλι αυξάνονται τα false positives. Για αυτό ο κατάλληλος αριθμός hash functions προσδιορίζεται καλύτερα πειραματικά για κάθε γράφο. Η επίδοση εξαρτάται έντονα από τη δομή του γράφου. Γράφοι με λιγότερα εναλλακτικά μονοπάτια και μικρότερο branching δίνουν καλύτερη ιχνηλάτηση, γιατί περιορίζουν την εξάπλωση των false positives στην ανάστροφη αναζήτηση. Αντίθετα, γράφοι με πολλούς convergence nodes και πολλές διακλαδώσεις δυσκολεύουν σημαντικά την αναγνώριση της σωστής πηγής.

Τελικό Συμπέρασμα

Η εργασία μας βοηθά να καταλάβουμε τις τρεις βασικές αλγοριθμικές τεχνικές για επεξεργασία δεδομένων μεγάλης κλίμακας.

Μετρητές Morris: Επιτρέπουν εκτίμηση του πλήθους χρησιμοποιώντας πολύ μικρό χώρο μνήμης. Η χρήση πολλών ανεξάρτητων μετρητών μειώνει τη διακύμανση, ενώ η επιλογή του α επηρεάζει σημαντικά την απόδοση για περιορισμένο αριθμό bits.

Flajolet-Martin και BJKST: Ο FM είναι απλός και πολύ ελαφρύς, αλλά δίνει χοντρική εκτίμηση χωρίς ρυθμιζόμενη ε-ακρίβεια. Ο BJKST δίνει πιθανοτικές εγγυήσεις με ρυθμιζόμενο ϵ , και με το median trick μπορεί να αυξήσει σημαντικά την πιθανότητα επιτυχίας.

Φίλτρα Bloom: Επιτρέπουν αποδοτικό υπολογισμό τομής συνόλων με μικρή επικοινωνία. Οι πολλαπλοί γύροι βελτιώνουν το αποτέλεσμα, ενώ στην ιχνηλάτηση πακέτων η απόδοση εξαρτάται έντονα από τη δομή του γράφου.

Τέλος το κοινό στοιχείο όλων των τεχνικών είναι ότι δέχονται μικρό πιθανοτικό σφάλμα ώστε να πετυχαίνουν σημαντική εξοικονόμηση μνήμης ή επικοινωνίας. Για αυτό είναι ιδιαίτερα χρήσιμες σε εφαρμογές μεγάλων ροών δεδομένων και κατανεμημένων συστημάτων.

